VisaBor / VisaCRM

Архитектурный отчёт: текущее состояние, достижения и открытые вопросы

v2.0 — 9 марта 2026 — Controlled Production Pilot

Содержание

1. Обзор проекта
2. Оценка зрелости (честная)
3. Архитектура и организация кода
4. Модульная структура
5. Безопасность: что сделано и что ещё нужно
6. Тестирование: покрытие и ограничения
7. Производительность: меры и отсутствие метрик
8. Биллинг: архитектура и открытые вопросы
9. Observability
10. Архитектурный долг: что закрыто, что осталось
11. Масштабируемость: гипотезы и необходимые проверки
12. Риски и план митигации
13. Disaster Recovery: текущий пробел
14. Дорожная карта до production GA

15. Итоговый вердикт
16. Полный бэклог (Wave 1-3 + ИИ + каналы лидов)

1. Обзор проекта

Два бренда — один бэкенд

Бренд	Назначение	URL	Аудитория
VisaBor	B2C маркетплейс/лендинг для клиентов	visabor.uz	Соискатели виз
VisaCRM	B2B SaaS CRM для визовых агентств	visacrm.uz (план)	Агентства, менеджеры
Owner Panel	Суперадмин-панель	visabor.uz/crm	Оператор платформы

15
Доменных модулей (организационных)

259
API-маршрутов

159
PHPUnit тестов

474
Assertions

Стек технологий

Компонент	Технология	Версия
Backend	Laravel	12 (PHP 8.4)
Frontend	Vue.js 3 + Tailwind CSS	3.x
База данных	PostgreSQL	16
Кэш / Очереди	Redis + Laravel Horizon	—
Аутентификация	JWT (phpopensourcesaver/jwt-auth)	—
SMS	Eskiz.uz stub	PIN=6236
Платежи	Manual без webhook	—
CI/CD	GitHub Actions	—
Мониторинг	Sentry + Health endpoint	—

Текущий статус

Проект находится на стадии **controlled production pilot**. Основной бэкенд, CRM-интерфейс, клиентский портал и Owner-панель функционируют. Однако SMS-верификация работает через заглушку (фиксированный PIN), платежи принимаются вручную (без webhook-интеграции с платёжными системами), OCR документов отсутствует. Это означает, что система пригодна для ограниченного пилотного запуска с контролируемым набором агентств и сценариев, но не для полноценного GA (General Availability) без доработки критичных интеграций.

2. Оценка зрелости (честная)

Область	Зрелость	Комментарий
Архитектура кода	Хорошо	Сервисный слой, разделение ответственности, facade pattern, Resources/FormRequests
Tenant Isolation	Хорошо	Двухуровневая: Eloquent Global Scope + PostgreSQL RLS. Но RLS не покрыт integration-тестами на PostgreSQL
Авторизация	Базовая	4 Policy класса (Case, Client, Document, User), owner/manager/superadmin. Нет полной permission matrix
Тестирование	Средне	159 тестов на SQLite in-memory. Нет integration-suite на PostgreSQL, нет e2e, нет нагрузочных
Биллинг	Структура есть	Чистая архитектура (facade + 4 сервиса). Нет webhook, идемпотентности, reconciliation
Observability	Базовая	Health (liveness), structured logs, Sentry. Нет readiness probe, APM, нет метрик p95/p99
SMS / Платежи	Заглушки	SMS stub PIN=6236, платежи manual. Критично для GA
Документы / OCR	Отсутствует	Загрузка/хранение работает. OCR — нет. Для визового продукта это core, не десерт
Disaster Recovery	Не описан	Нет задокументированного RPO/RTO, нет тестов восстановления

API Governance	Нет	Нет OpenAPI spec, нет versioning strategy, нет контрактного тестирования

Визуальная шкала зрелости

Код и архитектура

75%

Безопасность

55%

Интеграции (SMS, платежи, OCR)

20%

Операционная готовность (DR, monitoring, runbooks)

30%

3. Архитектура и организация кода

Слои приложения

HTTP Request → Middleware (JWT, RLS, Plan, Role) → FormRequest (валидация) → Controller (тонкий) → Policy (авторизация) → Service (бизнес-логика) → Model (Eloquent + Scope) → Resource (ответ)

Мультитенантность: двухуровневая изоляция

Уровень	Механизм	Что делает	Ограничения
Application	HasTenant trait + Global Scope	Все Eloquent-запросы автоматически фильтруются по agency_id текущего пользователя	Работает только через Eloquent. Raw SQL обходит scope
Database	PostgreSQL RLS policies	Даже при raw SQL запросе БД возвращает только данные текущего тенанта	Зависит от корректного SET context. Не тестируется в CI (SQLite)

Честное замечание о RLS

RLS — мощный защитный механизм, но его эффективность зависит от корректной настройки: запрет bypassrls для application-роли, отсутствие superuser-подключений из приложения, правильная установка session context. В текущем CI RLS не тестируется (тесты на SQLite), поэтому корректность RLS-политик подтверждена только ручной проверкой на production PostgreSQL, а не автоматически.

Рекомендация: добавить отдельный integration test suite на реальном PostgreSQL с проверкой RLS, session context, raw SQL access и отрицательными кейсами (попытка доступа к чужим данным через raw query).

Маршрутизация

259 маршрутов разделены на 4 файла:

Файл	Зона	Middleware
`routes/api/auth.php`	Регистрация, логин, refresh, logout	guest / auth:api
`routes/api/crm.php`	CRM агентства (cases, clients, documents, users)	auth:api + set.tenant + check.plan
`routes/api/owner.php`	Owner Panel (суперадмин)	auth:api + role:superadmin
`routes/api/public.php`	Публичный портал клиентов	auth.public (SMS OTP)

Отсутствие API versioning

Все маршруты живут под `/api/v1/`, но формальной стратегии версионирования нет. При едином бэкенде для B2C, B2B и Owner Panel без контрактного управления изменение API для одного интерфейса может сломать другой. OpenAPI-спецификация ещё не создана. При росте команды это станет болезненным.

4. Модульная структура

15 доменных папок

Модуль	Содержимое	Ключевая ответственность
Agency	Model, Service, DTO, Events	Регистрация, настройки, plan, рабочие страны

Auth	Service, DTOs, Controllers	JWT-аутентификация CRM
User	Model, Policy, Controller, Resource	Пользователи агентства (owner/manager)
Client	Model, Policy, Service, Events	Клиенты, семья, scoring
Case	Model, Policy, Service, Jobs, Events	Визовые заявки, SLA, stage flow
Document	Model, Policy, Controller	Загрузка/хранение документов
Payment	BillingEngine (facade), 4 сервиса	Подписки, платежи, wallet, ledger
Scoring	Service, Events	Оценка шансов по 5 блокам + red flags
PublicPortal	Controllers, Auth (SMS OTP)	Клиентский портал
Owner	Controllers	Суперадмин-панель, аналитика
TelegramBot	Controllers	Telegram webhook (заготовка)
Service	Model, Controller	Глобальные услуги, пакеты
Group	Model, Controller	Группировка заявок
Notification	Listeners	Обработка событий, activity log
Workflow	Stage transitions config	Конфигурация переходов этапов

Про "независимость" модулей

Модули организованы как доменные папки с собственными Models, Controllers, Services, Events, Policies, Resources, Requests. Это хорошо как организационная

структура кода и значительно лучше, чем монолитная свалка.

Однако это не доказанные bounded contexts с контрактными зависимостями и слабым сцеплением. Модули ссылаются друг на друга напрямую (Service одного модуля инжектит Model другого). Правильнее называть это **"доменно-организованный код"**, а не "независимые модули". Для настоящей модульной архитектуры нужны: явные публичные API модулей, запрет прямых cross-module зависимостей, контрактные интерфейсы и event-driven communication между bounded contexts.

5. Безопасность: что сделано и что ещё нужно

Что реализовано

Мера	Реализация	Статус
Аутентификация	JWT с TTL, refresh, blacklist	Работает
Tenant Isolation (App)	HasTenant global scope, auto agency_id	Работает
Tenant Isolation (DB)	PostgreSQL RLS policies	Работает, не тестируется в CI
Authorization	4 Laravel Policies (Case, Client, Document, User)	Работает + тесты
Input Validation	8 FormRequest классов с кастомными правилами	Работает

PII Filtering	API Resources скрывают passport, date_of_birth по умолчанию	Работает
Password Hashing	bcrypt	Стандарт
Auth Logging	Все попытки входа (success/fail) в auth channel	Работает
Rate Limiting	Laravel throttle middleware	Работает
HTTPS	Nginx SSL termination	Работает
CORS	Laravel CORS middleware	Работает
.env Protection	Не в git, server-only	Стандарт

Авторизационная модель

Policy	Действие	owner	manager	superadmin
CasePolicy	view	Все в агентстве	Только assigned или managers_see_all_cases	Все
	update / moveStage	Все	Только assigned	Все
	cancel	Да	Нет	Да
	delete	Да	Нет	Да
	reassign	Да	Нет	Да
ClientPolicy	view	Все	Все (same agency)	Все

	update	Все	Только с assigned cases	Все
	delete	Да	Нет	Да
DocumentPolicy	upload	Все	Assigned case	Все
	updateStatus	Все	Assigned case	Все
	delete	Да	Нет	Да
UserPolicy	viewAny / create	Да	Нет	Да
	update	Да	Нет	Да
	delete	Да (не себя, не owner)	Нет	Да

Неполнота авторизационной модели

Текущие Policies покрывают 4 основные сущности. Но для CRM с документами, финансами, группами, public portal и Owner Panel остаются вопросы без явного ответа:

- Кто видит полные PII (паспорт, дату рождения) — сейчас скрыты для всех через Resource, но нет Policy на "раскрытие"
- Кто может экспортировать данные (ZIP, CSV)?
- Кто меняет country requirements и service packages?
- Кто видит платёжную информацию других агентств (Owner Panel)?
- Кто может impersonate клиента на портале?
- Governance на уровне полной permission matrix отсутствует

Что НЕ реализовано (security gaps)

Открытые вопросы безопасности

Область	Текущее состояние	Что нужно	Приоритет
At-rest encryption	Паспортные данные хранятся в БД без шифрования полей	Шифрование чувствительных полей (passport_number, date_of_birth) на уровне application или column-level encryption	Высокий
Файловая безопасность	Документы хранятся на диске в storage/, доступ через контроллер	Signed URLs, private storage, antivirus/content scanning загрузок, миграция на S3 с server-side encryption	Высокий
Secret rotation	.env с фиксированными ключами	KMS или vault, ротация JWT secret, DB credentials	Средний
Audit trail на PII	activity_log через Spatie (события), auth logging	Логирование просмотра PII (кто когда смотрел паспорт клиента)	Средний
Data retention / deletion	SoftDeletes, но нет явной retention policy	Автоматическое удаление/ анонимизация персональных данных по истечении срока (GDPR-like)	Средний

Backup encryption	Не задокументировано	Encrypted backups, проверка восстановления	Средний

6. Тестирование: покрытие и ограничения

Текущее покрытие: 159 тестов, 474 assertions

Suite	Тестов	Assertions	Что проверяет
CaseService	20	—	Создание, лимиты плана, ВСЕ переходы этапов, отмена, завершение, полный путь
BillingEngine	21	—	Подписки, купоны, НДС, лимиты, смена плана, earn-first
API Endpoints	14	—	CRUD клиентов/заявок, middleware (роли, план, auth)
Auth	11	—	Регистрация, логин, me, refresh, logout, валидация
Policies	13	—	CasePolicy, ClientPolicy, UserPolicy — owner vs manager vs self
SLA	10	—	critical_date, applyStageSla, markOverdue, approachingDeadline

Tenant Isolation	7	—	Cross-tenant block, auto agency_id, scope filtering
Unit (Scoring, Helpers)	60+	—	Score calculation, Support helpers

Инфраструктура тестов

- **Engine:** PHPUnit + SQLite in-memory (быстро: ~3.5 секунды)
- **Фабрики:** `database/factories/Modules/{Agency,User,Client,Case,Payment}/`
- **Helpers:** createAgencyWithOwner(), createManager(), createClient(), authHeaders(), actingAsUser()
- **CI:** GitHub Actions — автозапуск при push/PR

Ключевое ограничение: SQLite vs PostgreSQL

Все 159 тестов выполняются на **SQLite in-memory**. PostgreSQL-специфичный SQL обёрнут в driver-check условия, RLS-политики и `SET LOCAL` обходятся условием `if (driver !== 'sqlite')`.

Для обычного Laravel CRUD это приемлемый trade-off (скорость vs точность). Но для системы, где один из ключевых security pillars — именно PostgreSQL RLS, это означает, что **главный защитный механизм не тестируется автоматически**.

Рекомендация:

- Добавить отдельный integration test suite на реальном PostgreSQL (можно в Docker)
- Тесты: включённый RLS, проверка session context, raw SQL access, отрицательные кейсы (попытка доступа к чужим данным)
- Запускать как отдельный job в CI (не блокирует основной suite)

Что не покрыто тестами

- E2E тесты (browser-based)
- Нагрузочное тестирование
- PostgreSQL RLS integration tests
- Contract tests для API (нет OpenAPI spec)
- Concurrency tests для биллинга (double-charge, race conditions)

7. Производительность: меры и отсутствие метрик

Реализованные меры

Мера	Реализация	Где
Redis-кэш	AgencyCacheService — dashboard (3 мин), kanban (2 мин), financial (5 мин)	DashboardController, CaseController
Event-driven инвалидация	CaseCreated, CaseStatusChanged, ClientRegistered, PaymentReceived → cache flush	Listeners
Composite индексы (7 шт)	cases(agency+stage+status), cases(agency+critical+stage), payments(agency+status+created), и др.	Migrations
Eager loading	with() в queries для предотвращения N+1	Controllers, Services
Пагинация	Все list-endpoints — paginated	Controllers

Queue processing	Laravel Horizon (Redis), SLA/billing jobs в фоне	Supervisor
Slow query detection	DB::listen >500ms → slow-queries.log (production only)	AppServiceProvider

Отсутствие метрик и бенчмарков

Перечисленные меры — это правильные архитектурные решения, но их эффективность **не доказана измерениями**.

На данный момент отсутствуют:

- Baseline latency (p50/p95/p99) для ключевых эндпоинтов
- Cache hit ratio (какой % запросов реально обслуживается из кэша)
- CPU/RAM/IO профиль под нагрузкой
- Результаты нагрузочного тестирования (k6, artillery, или хотя бы ab)
- Сравнение warm cache vs cold cache
- Benchmark read-heavy vs report-heavy сценариев

Утверждения о конкретных процентах снижения нагрузки или миллисекундах ответа невозможно сделать без этих данных. **Архитектура спроектирована с запасом, но доказательство — только нагрузочное тестирование.**

8. Биллинг: архитектура и открытые вопросы

Архитектура: Facade + 4 сервиса

BillingEngine (130 строк) — фасад, делегирующий в специализированные сервисы:

BillingEngine (facade) → SubscriptionService

BillingEngine (facade) → PlanChangeService

BillingEngine (facade) → PaymentProcessingService

BillingEngine (facade) → BillingHelperService

4 тарифных плана

План	Стоимость	Лимит заявок	Менеджеров	Модель
Free	0	5	1	—
Starter	$29/мес	50	3	Prepaid
Pro	$79/мес	200	10	Prepaid
Enterprise	$199/мес	Безлимит	Безлимит	Earn-first

Возможности биллинга

- **Upgrade / Downgrade:** pro-rata credit при upgrade, отложенный downgrade (в конце периода)
- **Earn-first модель (Enterprise):** платформа зарабатывает % от платежей клиентов агентства
- **Wallet / Ledger:** внутренний баланс, все операции записываются в ledger
- **Купоны:** процентные и фиксированные скидки
- **Аддоны:** дополнительные менеджеры, кейсы сверх лимита

- **НДС:** расчёт по стране агентства
- **Invoices:** автоматическая генерация при подписке
- **Expired subscriptions:** фоновый job (ежедневно 01:00)
- **Pending downgrades:** фоновый job (ежедневно 00:05)

Открытые вопросы биллинга

Архитектура чистая, тесты покрывают основные сценарии (21 тест). Но для финансовой системы под боем есть критичные пробелы:

Вопрос	Статус	Риск
Webhook-интеграция (Stripe/Payme)	Нет	Платежи принимаются вручную. Автоматическое подтверждение оплаты невозможно
Идемпотентность обработки событий	Не реализовано	Повторный webhook может привести к double-charge
Reconciliation	Нет	Нет механизма сверки внутреннего ledger с платёжной системой
Unique constraints на external payment IDs	Не проверено	Защита от дублей на уровне БД
Freeze на закрытый период	Нет	Можно задним числом модифицировать ledger
Concurrency tests	Нет	Race conditions при одновременных операциях на одном wallet
Accounting invariant tests	Частично	Есть тесты на баланс, но нет проверки "сумма всех операций = текущий баланс" как инвариант

9. Observability

Текущая реализация

Компонент	Реализация	Retention
Health endpoint	`GET /api/health` — DB, Redis, disk	—
Error tracking	Sentry	По настройкам Sentry
Auth logging	Все попытки входа (success/fail) в `auth` channel	30 дней
Billing logging	Платежи, подписки в `billing` channel	90 дней
Slow queries	>500ms → `slow_queries` channel (production)	14 дней
Audit trail	Spatie activity_log + audit channel	90 дней
Business events	11 Events → Listeners → structured logs	—

Events и Jobs

Event	Источник	Listener действие

CaseCreated	CaseService::create	Activity log, cache invalidation
CaseStatusChanged	CaseService::moveStage	Activity log, cache invalidation, structured log
CaseAssigned	CaseController::reassign	Activity log
SlaViolation	SendSlaWarningsJob	Structured log warning
ClientRegistered	ClientService::create	Structured log, cache invalidation
ClientCreatedViaPortal	PublicPortal	Structured log
PaymentReceived	PaymentController	Billing log, cache invalidation
DocumentUploaded	DocumentController	Structured log
ScoringCalculated	ScoringService	Structured log
SubscriptionChanged	BillingEngine	Billing log
SubscriptionExpired	ProcessExpiredSubscriptionsJob	Billing log warning

Background Jobs

Job	Расписание	Что делает
SendSlaWarningsJob	Каждые 15 мин	Проверяет critical_date, диспатчит SlaViolation для просроченных
ProcessExpiredSubscriptionsJob	Ежедневно 01:00	Обрабатывает истёкшие подписки

GenerateDashboardCacheJob	По событию	Обновляет кэш дашборда

Ограничения health-check

Текущий `/api/health` проверяет только liveness (DB подключение, Redis ping, disk space). Это полезно для UptimeRobot/Pingdom, но недостаточно для оценки реального состояния бизнес-логики.

Рекомендации:

- Разделить на `/health/live` (liveness) и `/health/ready` (readiness)
- Readiness: проверка Horizon, проверка последнего SLA job выполнения, smoke-check ключевых зависимостей
- Synthetic transaction: создание и удаление тестовой записи для проверки полного pipeline

10. Архитектурный долг: что закрыто, что осталось

Закрыты 7 выделенных блоков

Блок 1: Тесты Done

159 тестов, 474 assertions. CaseService, Auth, TenantIsolation, BillingEngine, API Endpoints, SLA, Policies.

Блок 2: Performance Done

Redis-кэш (3 слоя), event-driven invalidation, 7 composite индексов, Horizon, scheduled jobs.

Блок 3: API Resources Done

5 Resource классов, 8 FormRequest, PII скрыт по умолчанию, agency_id/deleted_at/password не возвращаются.

Блок 4: Рефакторинг Done

DashboardController 845→346 (5 сервисов), BillingEngine 883→130 (facade + 4 сервиса), routes 4 файла.

Блок 5: Policies Done

4 Policy класса, AuthorizesRequests в Controller, authorize() в контроллерах, 13 тестов.

Блок 6: Events/Jobs Done

11 Events, 3 Jobs (SLA warnings, expired subscriptions, dashboard cache), Listeners + activity_log.

Блок 7: Observability Done

Health endpoint, slow queries, 4 лог-канала (billing, auth, slow_queries, audit), auth logging.

Что осталось за рамками 7 блоков

Важное уточнение

Закрытие 7 выделенных блоков — это значительное улучшение кодовой базы. Но это не означает "100% архитектурный долг закрыт". Архитектурный долг — это не чекбокс, а хроническое состояние. Ниже — то, что осталось.

Задача	Статус	Приоритет для GA
Реальная SMS-интеграция (Eskiz.uz)	Заглушка	Критично
Платёжные webhook (Stripe/Payme)	Нет	Критично

OpenAPI спецификация	Нет	Высокий
PostgreSQL integration tests	Нет	Высокий
Нагрузочное тестирование	Нет	Высокий
At-rest encryption для PII	Нет	Высокий
Disaster recovery план	Нет	Высокий
OCR документов	Нет	Высокий*
Telegram webhook	Заготовка	Средний
visacrm.uz домен	Не настроен	Средний
Permission matrix (полная)	Частично	Средний
API versioning strategy	Нет	Средний
Billing: идемпотентность, reconciliation	Нет	Высокий (при webhook)

*** OCR:** Для визового продукта обработка паспортов и документов — это часть core workflow, а не "приятный десерт". В текущем MVP ставка сделана на ручную обработку документов менеджерами. Это осознанное решение для пилота, но для GA с масштабированием OCR становится высокоприоритетной задачей.

11. Масштабируемость: гипотезы и необходимые проверки

Целевые параметры

Параметр	Целевое значение
Агентств	200
Заявок в год	50 000
Одновременных пользователей	~200-400
Документов	~150 000 (3 на заявку)

Архитектурные решения, работающие на масштаб

- **Redis-кэш:** снижает нагрузку на БД для read-heavy дашбордов (TTL 2-5 мин)
- **Composite индексы:** 7 индексов на часто фильтруемые комбинации (agency+stage+status и др.)
- **Background jobs:** SLA-проверки и billing не блокируют HTTP-запросы
- **Пагинация:** все list-endpoints возвращают постраничные результаты
- **Eager loading:** предотвращает N+1 проблемы
- **Tenant scoping:** каждый запрос фильтрован по agency_id — объём данных в query пропорционален размеру агентства, а не всей платформы

Честная оценка

Архитектура **спроектирована с запасом** для заявленных параметров. Принятые решения (кэш, индексы, очереди, tenant scoping) — это стандартные и проверенные подходы для подобных нагрузок.

Однако **это гипотеза, а не доказанный факт**. Для подтверждения необходимо:

- Нагрузочное тестирование (k6/artillery) с реалистичными сценариями
- Профилирование CPU/RAM/IO под нагрузкой
- Измерение p95/p99 latency для ключевых эндпоинтов
- Проверка поведения при 200 агентствах с реальным объёмом данных
- Тестирование cache hit ratio и warm/cold cache сценариев

Вероятный сценарий: при росте потребуются read replica для аналитических запросов, S3/CDN для документов, materialized views для тяжёлых отчётов, отдельный queue server. Это нормально — но означает, что "без изменений инфраструктуры" — слишком категоричное утверждение.

12. Риски и план митигации

Риски, закрытые к моменту пилота

Риск	Как закрыт
Cross-tenant data leak	Двухуровневая изоляция (Eloquent scope + RLS)
God-object controllers	Рефакторинг в сервисы (Dashboard 845→346, BillingEngine 883→130)
Отсутствие авторизации на уровне сущностей	4 Policy класса + authorize() в контроллерах
PII leak через API	API Resources скрывают чувствительные поля по умолчанию
Нет мониторинга	Health endpoint + Sentry + structured logging

Нет фоновой обработки	3 Jobs: SLA, expired subscriptions, cache
Нет тестов	159 тестов, CI через GitHub Actions

Открытые риски

Риск	Влияние	Вероятность	Митигация (план)
SMS-заглушка в production	Высокое	100%	Подключить Eskiz.uz API до GA. В пилоте — контролируемый набор пользователей
Отсутствие платёжных webhook	Высокое	100%	Manual payments для пилота. Stripe/Payme интеграция до GA
RLS не тестируется в CI	Среднее	Среднее	Добавить PostgreSQL integration test suite
Нет DR плана	Высокое	Низкое (пилот)	Задокументировать RPO/RTO, настроить encrypted backups, протестировать восстановление
Паспортные данные без at-rest encryption	Высокое	Среднее	Column-level encryption для passport_number, date_of_birth
Документы на диске без защиты	Среднее	Среднее	S3 с server-side encryption, signed URLs, content scanning

Нет API versioning	Среднее	При росте команды	OpenAPI spec + контрактное тестирование + deprecation policy
Double-charge при конкурентных операциях	Высокое	Низкое (manual payments)	Идемпотентность, unique constraints на external payment IDs, accounting invariant tests
Single server SPOF	Высокое	При росте	Load balancer + минимум 2 app servers, managed DB

13. Disaster Recovery: текущий пробел

Состояние: не задокументировано

Для SaaS с документами и платежами disaster recovery — обязательный раздел. На данный момент:

Параметр	Текущее состояние	Рекомендация
RPO (Recovery Point Objective)	Не определён	Установить RPO ≤ 1 час для данных, ≤ 24 часа для документов
RTO (Recovery Time Objective)	Не определён	Установить RTO ≤ 4 часа

Database backups	Вероятно есть (VPS provider)	Автоматический pg_dump + encrypted offsite backup (S3)
Backup verification	Нет	Ежемесячный тест восстановления из backup
Rollback strategy	git revert + deploy	Blue/green или canary deploy, database migration rollback plan
Runbooks	Нет	Документация: что делать при падении DB, Redis, disk full, certificate expiry
Документы (файлы)	Только на диске сервера	S3 + cross-region replication

14. Дорожная карта до Production GA

Текущая стадия: Controlled Production Pilot

Система работает с ограниченным набором агентств, SMS через заглушку, платежи вручную.

Фаза 1: Критичные интеграции (до GA)

Задача	Зависимости	Блокирует GA

SMS: подключить Eskiz.uz API	API ключ, тестирование	Да
Платежи: Stripe или Payme webhook	Merchant account, идемпотентность	Да
At-rest encryption для PII	Column-level encryption, key management	Рекомендовано
DR plan: backups, RPO/RTO, runbooks	Инфраструктура	Рекомендовано

Фаза 2: Качество и уверенность

Задача	Цель
PostgreSQL integration tests	Подтвердить RLS работает корректно в CI
Нагрузочное тестирование	Получить реальные p95/p99, подтвердить или скорректировать гипотезы о масштабе
OpenAPI спецификация	Контрактное управление API, автогенерация клиентов
Permission matrix	Полная матрица: кто что видит и может делать

Фаза 3: Масштабирование

Задача	Когда нужно
OCR документов	При масштабировании обработки заявок
S3 + CDN для документов	При >50K документов или multi-server
Read replica для аналитики	При >100 агентств с тяжёлыми отчётами
Отдельный queue server	При >200 агентств
Telegram webhook	Для push-нотификаций клиентам

15. Итоговый вердикт

Что сделано хорошо

- Система перешла от "контроллер на стероидах" к зрелой сервисной архитектуре
- Двухуровневая tenant isolation (Eloquent scope + PostgreSQL RLS) — серьёзный defence-in-depth
- BillingEngine рефакторирован из монолита в facade с 4 специализированными сервисами
- 4 Policy класса обеспечивают гранулярную авторизацию owner/manager
- 159 тестов покрывают ключевые бизнес-сценарии и запускаются в CI
- Event-driven архитектура с 11 событиями и фоновыми задачами
- Structured logging по доменным каналам (auth, billing, slow queries)
- Redis-кэш с event-driven инвалидацией для read-heavy операций
- Все 7 выделенных блоков архитектурного долга закрыты

Что требует внимания до GA

- SMS и платежи — заглушки, критично для полноценного запуска
- RLS не тестируется в CI — главный защитный механизм не подтверждён автоматически
- Отсутствуют нагрузочные тесты — масштабируемость остаётся гипотезой
- Нет at-rest encryption для паспортных данных
- Disaster recovery не задокументирован
- OpenAPI и API versioning отсутствуют
- OCR документов — core workflow для визового продукта, пока ручная обработка

Формулировка статуса

Проект готов к **controlled production pilot** — ограниченному пилотному запуску с контролируемым набором агентств и сценариев. Инженерная база сильная: чистая архитектура, тестовое покрытие, мультитенантная изоляция, авторизация на уровне сущностей, event-driven обработка и observability.

Для перехода к **General Availability** необходимы: реальные SMS и платёжные интеграции, PostgreSQL integration tests, at-rest encryption для PII, disaster recovery план и нагрузочное подтверждение.

Закрытие 7 блоков архитектурного долга — значительный шаг. Но архитектурный долг — это не чекбокс, а хроническое состояние, которое требует постоянного внимания. Описанные рекомендации — следующие шаги на пути от "система работает" к "система доказала, что работает надёжно".

16. Полный бэклог (Wave 1-3 + ИИ + каналы лидов)

Wave 1 — Обязательно до GA Critical

Epic 1: Security & Compliance Foundation

#	Feature	Ключевые задачи	DoD
1.1	**Шифрование PII**	• Определить перечень PII-полей • At-rest encryption (passport_number, date_of_birth и др.) • Стратегия ротации ключей (KMS) • Model mutators/accessors, прозрачное чтение/запись • Тесты: поиск, фильтрация, экспорт с шифрованием	PII зашифрованы в БД, ключи не в коде, негативные тесты проходят
1.2	**Защита файлов**	• Private storage, signed URLs • MIME/type/content validation • Antivirus (начать с validation, позже ClamAV/облачный*) • Запрет прямой раздачи из public/ • Журнал скачивания/просмотра • Подготовка к S3 + server-side encryption **ClamAV ~1GB RAM. Начать с MIME, антивирус при S3*	Документы не по прямым ссылкам, заражённые отклоняются, аудит доступа

1.3	**Permission Matrix**	• Полная матрица: owner/manager/superadmin/client • Права на PII, экспорт, документы, финансы, скоринг, impersonation • Доработка policies/gates • Тесты на запрет/разрешение	Утверждённая матрица, все операции покрыты policy/gate

Epic 2: Core Integrations

#	Feature	Ключевые задачи	DoD
2.1	**SMS Eskiz.uz**	• Eskiz API интеграция • OTP lifecycle (одноразовый, TTL) • Rate limit, anti-bruteforce • Логирование отправок и статусов • Fallback policy, шаблоны SMS в конфиг	SMS в реальный шлюз, OTP одноразовый, rate limit работает

2.2	**Платежи webhook**	• Payme/Stripe webhook + валидация подписи • Идемпотентная обработка • Unique constraint на external payment id • webhook → invoice → ledger → wallet • Retry/dead-letter, reconciliation report	Платёж автоматически, двойная обработка исключена, reconciliation

Epic 3: Reliability

#	Feature	Ключевые задачи	DoD
3.1	**Backup & Restore**	• RPO ≤1ч (данные), ≤24ч (файлы) / RTO ≤4ч • pg_dump + encrypted offsite (S3) • Restore drill (ежемесячно) • Документация восстановления	Backup автоматический, restore протестирован, копии offsite

Epic 5: Testing

#	Feature	Ключевые задачи	DoD

5.1	**PostgreSQL RLS tests**	• PostgreSQL в CI (Docker) • RLS policies включены • Positive/negative cases, session context, raw SQL	RLS на PostgreSQL, cross-tenant блокируется автоматически
5.4	**Billing invariants**	• Double webhook tests • Race conditions для wallet • Invariant: сумма операций = баланс • Coupon/addon/proration, rollback	Ledger consistency, duplicate webhook безопасен

Wave 2 — Управляемая платформа High

#	Feature	Суть
3.2	**Runbooks**	Инструкции на DB/Redis/disk/queue/SSL/webhook сбои, эскалация, post-incident review
3.3	**Health-check v2**	/health/live + /health/ready, synthetic transaction, alerting
4.1	**OpenAPI**	Описание endpoints, request/response schemas, связь с CI, автогенерация docs
4.2	**API versioning**	Deprecation policy, contract tests, backward compatibility, migration playbook
5.2	**E2E тесты**	Playwright, критичные сценарии, smoke suite в CI

5.3	**Нагрузочное тестирование**	k6, baseline p50/p95/p99, capacity report, триггеры масштабирования
6.2	**Data lifecycle**	Сроки хранения, архивирование, anonymization, retention policy
7.1	**APM / Observability**	Tracing, метрики, alerting thresholds, operational dashboard

Wave 3 — Масштабирование Medium

#	Feature	Суть	Замечание
2.3	**Telegram webhook**	Уведомления по событиям, user preferences	—
2.4	**OCR документов**	Pipeline загрузка → OCR → review, confidence score	Core workflow для визового продукта. В MVP — ручная обработка (осознанно)
6.1	**Контракты модулей**	Boundaries, events, запрет cross-module	Over-engineering при 1-2 разработчиках. Актуально при 3+
7.2	**Масштабирование infra**	Read replica, queue server, S3/CDN	Infra roadmap с условиями включения

ИИ-функции для Enterprise Feature Pack

Бизнес-модель

- Свой API-ключ на сервере (ANTHROPIC_API_KEY)
- Enterprise ($99/мес) = 100 генераций/мес включено
- Доп. пакет: +500 генераций за $29/мес
- Себестоимость: Haiku ~$0.30/мес, Sonnet ~$2/мес → маржа 97-99%

Доступ по тарифам

Возможность	Trial/Starter	Pro	Enterprise
Шаблоны документов	—	3 базовых	Все
База знаний (глобальная)	—	Чтение	Чтение
База знаний (своя wiki)	—	—	Свои + редактирование
Онбординг менеджеров	—	Да	Да
ИИ-генерация документов	—	—	100/мес
ИИ-проверка пакета	—	—	100/мес
ИИ-памятка клиенту	—	—	100/мес
ИИ-оценка рисков	—	—	100/мес

Этапы реализации

Этап	Содержание

1. Инфраструктура ИИ	Таблицы ai_usage_log + ai_quota, AiService (Claude API), middleware ai.quota, покупка доп. пакета
2. Генерация документов	Шаблоны в БД, cover letter, справка с работы, спонсорское, визовые анкеты (DS-160, Шенген, UK), PDF
3. ИИ-анализ	Проверка пакета документов, оценка рисков отказа, персональная памятка, рекомендации по усилению
4. База знаний	Глобальная wiki (суперадмин), wiki агентства (Enterprise), онбординг менеджеров, шаблоны процессов

Внешние каналы лидов (Этап 5) Feature Pack

Канал	Starter	Pro	Enterprise
Ручное создание	Да	Да	Да
Маркетплейс VisaBor	Да	Да	Да
API-ключ для внешних лидов	—	Да	Да
JS-виджет для сайта	—	Да	Да
Instagram Lead Ads (Meta)	—	—	Да
Zapier/Make интеграция	—	—	Да

Задачи: API-ключ агентства, POST /external/leads, JS-виджет, Instagram Lead Ads (Meta OAuth), Zapier/Make webhook, поле lead_source, дашборд по источникам.

Минимальные критерии GA

#	Критерий	Статус
1	Реальная SMS работает	Заглушка
2	Webhook-платежи идемпотентны	Нет
3	RLS integration suite зелёный	Нет
4	Backup/restore проверен	Нет
5	PII и файлы защищены	Частично
6	Permission matrix утверждена	Частично (4 Policy)
7	p95/p99 измерены и в норме	Нет метрик
8	OpenAPI baseline опубликован	Нет
9	E2E smoke suite стабилен	Нет

VisaBor / VisaCRM — Архитектурный отчёт v2.0 + Полный бэклог — 9 марта 2026

Ревизия: честная оценка состояния, без маркетинга